

Written Homework Assignment

1

Problem 1.1, Stephens page 13

What are the basic tasks that all software engineering projects must handle?

Requirements Gathering, High-Level Design, Low-Level Design, Development, Testing, Deployment, Maintenance, Wrap-up

Problem 1.2, Stephens page 13

Give a one sentence description of each of the tasks you listed in Exercise 1.

- Requirements Gathering: Figure out what the customer requires.
- High-Level Design: Outline the biggest, most important parts of the software.
- Low-Level Design: Outline in detail how to build the software so programmers can understand how to make it.
- Development: Write the actual code.
- Testing: Run the software in many different scenarios to find flaws.
- Deployment: Deploy the software to the users.
- Maintenance: Fix bugs, make improvements, and work on new additions.
- Wrap-up: Reflect on the project to learn what you can do better in the future.

Problem 2.4, Stephens page 27

Like Microsoft Word, Google Docs [sic] provides some simple change tracking tools. Go to <http://www.google.com/docs/about/> to learn more and sign up [if you do not have an account already]. Then create a document, open the File menu's Version History submenu, select Name Current Version, and name the file

'Version 1'. Make some changes and repeat the preceding steps to name the revised document 'Version 2'. Now open the File menu's Version History submenu again but this time select See Version History. Click the versions listed on the right to see what changed between versions. Document what you've noticed about the information you see, and how the differences between versions are displayed.

Investigation: Compare this process to what you can do with GitHub versions. How are the two tools different? How are they the same?

In Google Docs:

Timeline showing saved versions

Color coded changes

Restore option

Very frequent and detailed saves

Compared to GitHub:

Both have version history, tracking users changes, and named versions

GitHub allows multiple branches for development, and requires messages before every commit, while Docs updates every change instantly.

Problem 2.5, Stephens page 27

What does JBGE stand for and what does it mean?

1. Just Barely Good Enough. It means you should only write documentation and comments that are necessary for understanding, and not waste time overwriting.

Data for Problems 4.2 and 4.4

Table 4.2 [below] summarizes some of the classes and modules you might need (and their unreasonably optimistic expected times) to develop players and zombies for the game. (The program would also need lots of other pieces not listed here to handle other parts of the game.)

Use the following table of data for Exercises 4.2 and 4.4.

Task	Time (Days)	Predecessor s
A. Robotic control module	5	—

B. Texture library	5	C
C. Texture editor	4	—
D. Character editor	6	A, G, I
E. Character animator	7	D
F. Artificial intelligence (for zombies)	7	—
G. Rendering engine	6	—
H. Humanoid base classes	3	—
I. Character classes	3	H
J. Zombie classes	3	H
K. Test environment	5	L
L. Test environment editor	6	C, G
M. Character library	9	B, E, I
N. Zombie library	15	B, J, O
O. Zombie editor	5	A, G, J
P. Zombie animator	6	O
Q. Character testing	4	K, M
R. Zombie testing	4	K, N

Problem 4.2, Stephens page 78

1. Use **critical path methods** to find the total expected time from the project's start for each task's completion.
2. Find the critical path. What are the tasks on the critical path?
3. What is the total expected duration of the project in working days?

The critical path is tasks G, D, E, M, and Q. The expected length is 32 working days.

Problem 4.4, Stephens page 78

Build a Gantt chart for the critical path you drew in Exercise 2. Start on Wednesday, January 1, 2024, and don't work on weekends or the following holidays:

Holiday	Date
New Year's Day	January 1
Martin Luther King Day	January 20
President's Day	February 17
St. Valentine's Day	February 14
Alien Overload	March 26
Appreciation Day	
Income Tax Day	April 15

Problem 4.6, Stephens page 79

In addition to losing time from vacation and sick leave, projects can suffer from problems that just strike out of nowhere, like a bad version of *deus ex machina*. For example, senior management could decide to switch your target platform from Windows desktop PCs to the latest smartwatch technology. Or a pandemic, hurricane, trade war, earthquake, alien invasion, and so on could delay the shipment of your new servers. [Not that anything as far-fetched as a pandemic might occur, right?] Or one of your developers might move to Iceland, which is a real nice place to raise your kids up. How can you handle these sorts of completely unpredictable problems?

You can handle these sorts of unpredictable problems similar to vacation and sick leave. Leave time near the end of the schedule to account for *deus ex machina* problems.

Problem 4.8, Stephens page 79

According to your textbook, what are the two biggest mistakes you can make while tracking tasks?

The first biggest mistake is not doing something when a task is falling behind.

The second biggest mistake is thinking that just throwing more people at a task will get it done faster.

Problem 5.1, Stephens page 114

List five characteristics of good requirements.

Clear, Unambiguous, Consistent, Prioritized, and Verifiable.

Problem 5.3, Stephens page 114

Suppose you want to build a program called TimeShifter to upload and download files at scheduled times while you're on vacation. The following list shows some of the applications requirements.

- a. Allow users to monitor uploads/downloads while away from the office. (B)
- b. Let the user specify website log-in parameters such as an Internet address, a port, a username, and a password. (U, F)
- c. Let the user specify upload/download parameters such a number of retries if there's a problem. (U, F)
- d. Let the user select an Internet location, a local file, and a time to perform the upload/download. (U, F)
- e. Let the user schedule uploads/downloads at any time. (N)
- f. Allow uploads/downloads to run at any time. (N)
- g. Make uploads/downloads transfer at least 8 Mbps. (N)
- h. Run uploads/downloads sequentially. Two cannot run at the same time. (N)
- i. If an upload/download is scheduled for a time whan another is in progress, it waits until the other one finishes. (N)
- j. Perform schedule uploads/downloads. (F)

- k. Keep a log of all attempted uploads/downloads and whether the succeeded. (F)
- l. Let the user empty the log. (U, F)
- m. Display reports of upload/download attempts. (U, F)
- n. Let the user view the log reports on a remote device such as a phone. (U, F)
- o. Send an e-mail to an administrator if an upload/download fails more than its maximum retry number of times. (U, F)
- p. Send a text message to an administrator if an upload/download fails more than its maximum return number of times. (U, F)

For this exercise, list the audience-oriented categories for each requirement. Are there requirements in every category? [If not, state why not...]

There are requirements in every category.

Problem 5.9, Stephens page 115

Figure 5-1 [right] shows the design for a simple hangman game that will run on smartphones. When you click the New Game button, the program picks a random mystery word from a large list and starts a new game. Then if you click a letter, either the letter is filled in where it appears in the mystery word, or a new piece of Mr. Bones's skeleton appears. In either case, the letter you clicked is grayed out so that you don't pick it again. If you guess all the letters in the mystery word, the game displays a message that says, "Congratulations, you won!" If you build Mr. Bones's complete skeleton, a message says, "Sorry, you lost."

Brainstorm this application and see if you can think of ways you might change it. Use the MOSCOW method to prioritize your changes.

Must Have

-Play again button after win/loss

Should have

-Timed rounds

-Points for speed

Could have

-Change skeleton character

-Dark/light mode

-Different visual styles

Won't have for now

-Profiles, saves, syncing across devices



FIGURE 4-1: The Mr. Bones application is a hangman word game for Windows Phone.